arXiv:2603.00338v2 [cs.RO] 10 Sep 2026

Layered Safety: Enhancing Autonomous Collision Avoidance via Multistage CBF Safety Filters

Erina Yamaguchi, Ryan M. Bena, Gilbert Bahati, and Aaron D. Ames

***Abstract*— This paper presents a general end-to-end framework for constructing robust and reliable layered safety filters that can be leveraged to perform dynamic collision avoidance over a broad range of applications using only local perception data. Given a robot-centric point cloud, we begin by constructing an occupancy map which is used to synthesize a *Poisson safety function* (PSF). The resultant PSF is employed as a *control barrier function* (CBF) within two distinct safety filtering stages. In the first stage, we propose a predictive safety filter to compute optimal safe trajectories based on nominal potentially-unsafe commands. The resultant short-term plans are constrained to satisfy the CBF condition along a finite prediction horizon. In the second stage, instantaneous velocity commands are further refined by a real-time CBF-based safety filter and tracked by the full-order low-level robot controller. Assuming accurate tracking of velocity commands, we obtain formal guarantees of safety for the full-order system. We validate the optimality and robustness of our multistage architecture, in comparison to traditional single-stage safety filters, via a detailed Pareto analysis. We further demonstrate the effectiveness and generality of our collision avoidance methodology on multiple legged robot platforms across a variety of real-world dynamic scenarios.**

I. INTRODUCTION

As the deployment of robotic systems continues to evolve from controlled laboratories to unstructured real-world environments, autonomous safe operation in dynamic settings becomes crucial. For legged robots and humanoids, collisions can compromise task objectives, necessitating the constructive and robust integration of perception, planning, and control under sensor noise and partial observations. This is particularly challenging when considering the task of collision avoidance using onboard sensing and computation, where the robot must continually acquire environmental occupancy information, process it into a reliable safety representation, and translate this into safe control actions.

On this front, control barrier function (CBF) methods [1], [2] provide a computationally efficient mechanism for enforcing real-time safety and have been demonstrated across many platforms [3]–[8]. However, most CBF-based methods operate myopically, relying on a quadratic program (QP) to optimize instantaneous safety, which can result in conservative behavior, nonpersistent feasibility, or task failure when immediate safe actions conflict with longer-term goals [9], [10]. Model predictive control (MPC) [11] mitigates this limitation by optimizing a future trajectory using safety-assured planning strategies [12], including enforcing discrete-time CBF constraints along a receding finite horizon [13]–[18]. However, MPC exhibits fragility due to uncertainties about future environments and the challenges in solving highly nonconvex optimization problems. Furthermore, most existing safety-critical control methodologies handle the specification of safety constraints *a priori*, typically through hand-designed geometric descriptions or signed distance fields [19]–[26]. In this paper, we address the real-time synthesis of safety constraints using the Poisson safety function (PSF) method [8], [27], which produces smooth functional safety representations directly from perception data, enabling systematic CBF synthesis. Critically, the PSF approach produces a dynamically-updated unified collision avoidance constraint.

High-dimensional dynamics add complexity when ensuring safety. For legged robots, most approaches rely on full-order dynamics [28], [29], typically rendering online planning and safety constraint synthesis computationally prohibitive and limiting reaction time. Furthermore, in practice, safety filters often lack access to the full-order control inputs, only passing reference commands to low-level controllers.

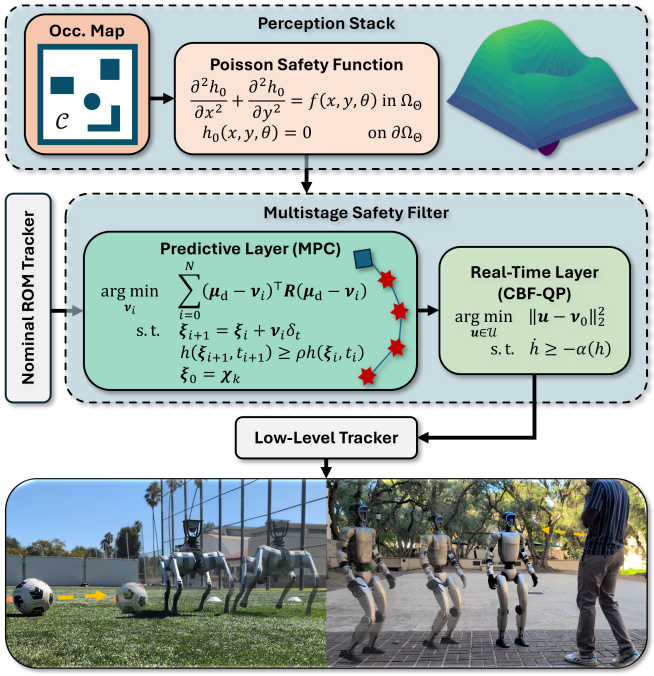

Fig. 1: Layered Safety Filter. The PSF is computed using an occupancy map safe set representation. The solution enables the construction of safety constraints for use in a multistage safety filter: a predictive layer in series with a real-time layer. This framework generates safe autonomous actions, leading to collision-free behavior around dynamic obstacles. Experimental footage provided at https://youtu.be/8v7uyheOM2k

This research is supported by BP.

Authors are with the Department of Mechanical and Civil Engineering and the Department of Aerospace, Caltech, Pasadena, CA 91125, USA, {erinay, ryanbena, gbahati, ames}@caltech.edu

To overcome these limitations, recent approaches employ learning-based policies [30] at the expense of formal safety guarantees. Instead, a layered architecture approach is introduced in [31] combining low-frequency prediction with high-frequency safety enforcement. However, their formulation relies on knowledge of platform-specific dynamics with a custom low-level controller. To enhance generality, we leverage reduced-order models (ROMs) [4], [5], [32]–[34] that capture dominant dynamics while enabling tractable synthesis of safety constraints. This viewpoint decouples safety synthesis from full-order modeling: safe commands are planned on the ROM and safety for the full-order dynamics follows under accurate tracking of the ROM commands by the robot, yielding a *model-free* architecture.

Building on these foundations, this work presents a multistage safety filter combining MPC-like predictive planning with CBF-QP real-time safety, embedded in an end-to-end perception-to-control architecture as illustrated in Fig. 1. In the proposed model-free layered pipeline, a predictive safety filter generates safe nominal commands, that are then refined by a real-time CBF-QP safety filter. This architecture unifies perception-based CBF synthesis with predictive and real-time CBF-based safety filtering, addressing a gap in existing approaches, typically emphasize only one part of this pipeline. Existing occupancy mapping methods provide rich environment representations but lack constructive interfaces for real-time CBF synthesis. ROM-based CBF approaches enable computationally efficient real-time safety filtering [4], [5], [32]–[34], but often do not incorporate perception or predictive planning. Predictive safety approaches account for long-horizon behavior, but become computationally expensive when combined with real-time safety under full-order dynamics [31]. Our contributions are summarized as follows:

- *Perception-based safety synthesis:* A real-time point-cloud-based occupancy mapping method for dynamic environments, enabling the synthesis of a PSF that serves as a CBF with geometric parameterization.
- *Multistage model-free control architecture:* A cascaded MPC-based predictive safety filter and a real-time CBF-QP safety filter on a ROM, with theoretical guarantees of accurate tracking by the full-order robot dynamics.
- *Performance analysis:* Characterization of performance optimality and robustness tradeoffs via a detailed Pareto optimality analysis, demonstrating the benefits of the layered approach over traditional single-stage methods.
- *Hardware verification & validation:* Extensive demonstrations on quadrupedal and humanoid platforms performing dynamic collision avoidance in real-world environments with fully-onboard sensing and computation.

The structure of the paper is as follows: Section II reviews CBFs for safety-critical control, PSFs for perception-based safety, and MPC. Section III introduces the proposed perception and multistage control framework. The multistage architecture is validated experimentally in Section IV. Finally, conclusions are in Section V.

II. Background

We consider nonlinear control affine systems of the form:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}, \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^n$ denotes the state vector, $\mathbf{u} \in \mathbb{R}^m$ represents the control input, $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^n$ denotes the drift dynamics, and $\mathbf{g} : \mathbb{R}^n \to \mathbb{R}^{n\times m}$ is the actuation matrix, both assumed to be locally Lipschitz continuous. A locally Lipschitz continuous controller, $\mathbf{k} : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}^m$, yields the closed loop system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{k}(\mathbf{x}, t)$ which, given an initial condition, admits a unique solution $t \mapsto \mathbf{x}(t)$ which we assume exists for all time for brevity [35].

A. Safety-Critical Control

We formalize the notation of safety through the lens of *forward invariance*, where system trajectories $t \mapsto \mathbf{x}(t)$ are required to remain in a *safe set* for all time. We consider time-varying safe sets defined as the 0-super level set of a function $h : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}$ denoted by:

$$\mathcal{C}_t = \{\mathbf{x} \in \mathbb{R}^n : h(\mathbf{x}, t) \geq 0\}. \tag{2}$$

CBFs provide a constructive framework for designing controllers that enforce forward invariance.

Definition 1. (Control Barrier Function [1]) Let $\mathcal{C}_t \subset \mathbb{R}^n$ be the time-varying 0-super-level set of a continuously differentiable function $h : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}$ satisfying $\nabla h(\mathbf{x}, t) \neq \mathbf{0}$ when $h(\mathbf{x}, t) = 0$. We call h a time-varying CBF for (1) if there exists[1] $\alpha \in \mathcal{K}^e_\infty$ such that for all $(\mathbf{x}, t) \in \mathbb{R}^n \times \mathbb{R}_+$:

$$\sup_{\mathbf{u}\in\mathbb{R}^m} \dot{h}(\mathbf{x}, t, \mathbf{u}) \coloneqq \nabla h(\mathbf{x}, t) \cdot (\mathbf{f}(\mathbf{x}) + \mathbf{g}(\mathbf{x})\mathbf{u}) + \frac{\partial h}{\partial t}(\mathbf{x}, t) \geq -\alpha(h(\mathbf{x}, t)). \tag{3}$$

Given a nominal (possibly unsafe) controller $\mathbf{k}_{\text{nom}} : \mathbb{R}^n \times \mathbb{R}_+ \to \mathbb{R}^m$, one way of constructing a safe controller is via the CBF-QP that adjusts $\mathbf{k}_{\text{nom}}$ to its nearest safe action:

$$\mathbf{k}(\mathbf{x}, t) = \underset{\mathbf{u}\in\mathbb{R}^m}{\arg\min} \quad \|\mathbf{u} - \mathbf{k}_{\text{nom}}(\mathbf{x}, t)\|_2^2 \qquad \text{(CBF-QP)}$$
$$\text{s.t.} \quad \dot{h}(\mathbf{x}, t, \mathbf{u}) \geq -\alpha(h(\mathbf{x}, t)).$$

B. Perception-based Safety with Poisson Safety Functions

While CBFs provide a principled tool for synthesizing controllers that guarantee safe actions, their success in robotic applications rely on the availability of a function h_0 to encode physical environmental safety specifications from perception data in spatial coordinates $\mathbf{y}(\mathbf{x}) = (x, y, z) \in \mathbb{R}^3$. Given occupancy information from perception, PSFs [8] provide such safety representations, bridging the gap between perception and CBF-based safety.

Definition 2. (Poisson Safety Function [8]) Given an occupancy map, let $\Omega \subset \mathbb{R}^3$ denote the open, bounded, and connected set of free space, with smooth boundary $\partial\Omega$

[1]A continuous function $\alpha : \mathbb{R} \to \mathbb{R}$ belongs to the extended class $\mathcal{K}^e_\infty$ if it is monotonically increasing, $\alpha(0) = 0$, $\lim_{s\to\infty} \alpha(s) = \infty$, and $\lim_{s\to-\infty} \alpha(s) = -\infty$.

corresponding to obstacle surfaces. A PSF, $h_0 : \mathbb{R}^3 \to \mathbb{R}$, is defined as the unique solution Dirichlet problem for Poisson's equation:

$$\begin{cases} \Delta h_0(\mathbf{y}) = f(\mathbf{y}) & \text{in } \Omega, \\ h_0(\mathbf{y}) = 0 & \text{on } \partial\Omega, \end{cases} \tag{4}$$

where $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2}$ is the *Laplacian* and $f : \Omega \to \mathbb{R}_{<0}$ is a given forcing function.

The boundary conditions in (4) assigns the 0-level set of h_0 to obstacle surfaces, while $f(\mathbf{y}) < 0$ guarantees $h_0(\mathbf{y}) > 0$ throughout the free space. As discussed in [8], a smooth forcing function $f \in C^\infty(\overline{\Omega})$, yields smooth solutions to (4), $h_0 \in C^\infty(\overline{\Omega})$. It follows that h_0 defines a CBF for systems with single-integrator dynamics (i.e., $h_0 := h$), and its smoothness allows for CBF extensions for higher-order systems [8]. For dynamic environments, (4) can be parametrized with time to account for temporal variations yielding a time-varying safety function $t \mapsto h_0(\mathbf{y}, t)$ [27].

C. Model Predictive Control

Controllers based on predictive optimization enable the parallel enforcement of both safety and performance goals. In our collision avoidance framework, we employ an MPC-based *predictive safety filter* that incorporates the discrete-time CBF (DCBF) as in [13], [36], enforcing safety over a time horizon. The following finite-time optimal control problem (FTOCP) with a DCBF constraint along a horizon of length $N \in \mathbb{N}_{\geq 1}$ defines the predictive safety filter:

$$\begin{aligned} \min_{\substack{\boldsymbol{\xi}_{0:N} \in \mathbb{R}^n \\ \boldsymbol{\nu}_{0:N-1} \in \mathbb{R}^m}} \quad & \sum_{i=0}^{N-1} c(\boldsymbol{\xi}_i, \boldsymbol{\nu}_i) + V(\boldsymbol{\xi}_N) \\ \text{s.t.} \quad & \boldsymbol{\xi}_{i+1} = \mathbf{F}(\boldsymbol{\xi}_i, \boldsymbol{\nu}_i), \quad \forall i \in \{0, \dots, N-1\} \\ & h(\boldsymbol{\xi}_{i+1}) \geq \rho h(\boldsymbol{\xi}_i), \quad \forall i \in \{0, \dots, N-1\} \\ & \boldsymbol{\xi}_0 = \boldsymbol{x}_k \end{aligned} \tag{5}$$

where $c : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$ is the stage cost, $V : \mathbb{R}^n \to \mathbb{R}$ is the terminal cost used to approximate the infinite-horizon optimal control problem, $\boldsymbol{\xi}_i \in \mathbb{R}^n$, $\boldsymbol{\nu}_i \in \mathbb{R}^m$ represent the planned state and inputs at discrete times, and $\mathbf{F} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ is the discretization of (1) over a finite time interval. The first constraint ensures the planned trajectory is dynamically feasible, the second constraint is the DCBF condition that enforces safety along the trajectory, and the last constraint ensures the plan aligns with the current state.

The optimal plan of inputs of this FTOCP is computed as $[\boldsymbol{\nu}_0^*(\boldsymbol{x}_k), \dots, \boldsymbol{\nu}_{N-1}^*(\boldsymbol{x}_k)]$, and $\boldsymbol{\nu}_0^*(\boldsymbol{x}_k)$ is applied to the system, yielding the MPC+CBF controller:

$$\boldsymbol{\pi}^{\text{MPC+CBF}}(\boldsymbol{x}_k) = \boldsymbol{\nu}_0^*(\boldsymbol{x}_k). \tag{6}$$

This $\boldsymbol{\pi}^{\text{MPC+CBF}}$ controller displays favorable performance and robustness properties when compared to standard MPC or CBF controllers [36] by enforcing the safety constraint in the FTOCP and optimizing across the planning horizon N.

III. Multistage Safety Filter: Architecture

In this section, we present a multistage safety framework that unifies onboard perception, CBF synthesis, and MPC for dynamic collision avoidance in robotic systems. Specifically, we build an occupancy map from point cloud data, construct a boundary value problem for Poisson's equation, and solve it to obtain a PSF. The resulting PSF serves as a CBF that is incorporated into both a *predictive* safety layer—enforcing safety over a receding horizon via MPC—and a *real-time* safety layer—enforcing instantaneous safety via a CBF-QP.

A. Occupancy Mapping

Robotic systems collect local environment occupancy information in real-time via onboard sensors. This information is used to estimate membership of a safe set, denoted as $\mathcal{C}_0 \subset \mathbb{R}^2$, containing all unoccupied points $(x, y) \in \mathbb{R}^2$ in local two-dimensional environments. For perception-in-the-loop applications, the true set $\mathcal{C}_0$ cannot be known exactly; however, a discretized approximation of $\mathcal{C}_0$ can often be obtained in the form of an occupancy map.

Let $\mathcal{S}$ be the set of grid cells in a local 2D occupancy map, and let $p \in \mathcal{S}$ be a single grid cell in the map corresponding to a fixed location in space. The occupancy map representing the true safe set $\mathcal{C}_0$ is the binary function $M : \mathcal{S} \to \{0, 1\}$:

$$M(p) = \begin{cases} 1, & \text{if cell } p \in \mathcal{S} \text{ is occupied,} \\ 0, & \text{if cell } p \in \mathcal{S} \text{ is free.} \end{cases} \tag{7}$$

Next, we compute the occupancy map $\hat{M}$, which estimates M from successive robot-centric point clouds.

1) Instantaneous Point Map (M_0): To construct $\hat{M}$, we first remove ground and self detections, crop unmapped regions, and project each remaining detection onto the instantaneous 2D occupancy map $M_0 : \mathcal{S} \to \{0, 1\}$ via:

$$M_0(p) := \begin{cases} 1, & \text{if } p \in \mathcal{S} \text{ contains a point detection,} \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

2) Spatially Convolved Map ($\bar{M}$): Generally, M_0 contains gaps and isolated cells that arise from noise, occlusion, and the limited angular resolution of perception-sensor scans. To improve spatial consistency, we convolve M_0 with an unnormalized compactly supported[2] kernel K:

$$\bar{M} := K * M_0, \tag{9}$$

where $*$ denotes discrete convolution operator. This operation smooths M_0 and amplifies regions of spatial consistency.

3) Dynamic Confidence Map (Γ): To enforce temporal continuity, we define a continuous *confidence map* $\Gamma : \mathcal{S} \to [0, 1]$, where $\Gamma(p)$ represents the confidence a cell $p \in \mathcal{S}$ is occupied. Given $\bar{M}$, the confidence dynamics are governed by the cell-wise continuous-time update law:

$$\dot{\Gamma}(p) := \begin{cases} -\beta^- \Gamma(p) & \text{if } \bar{M}(p) < \sigma, \\ \beta^+ \bar{M}(p)\,(1 - \Gamma(p)) & \text{if } \bar{M}(p) \geq \sigma, \end{cases} \tag{10}$$

[2]A kernel is *compactly supported* if it vanishes outside a finite radius r, so only neighboring cells within distance r contribute to the convolution.

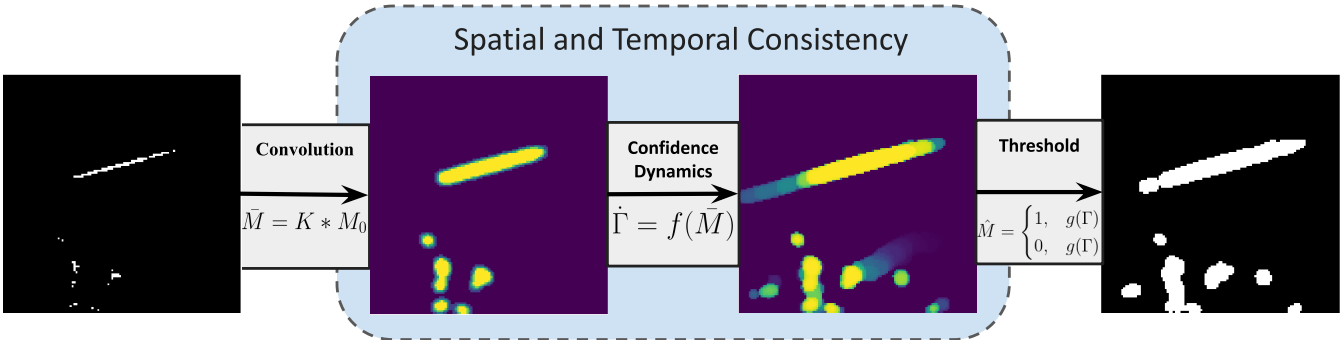

Fig. 2: Occupancy Mapping. **Left** First, we project each point cloud onto a 2D grid M_0. **Middle Left** Second, we convolve M_0 with a Gaussian kernel K, adding weight to occupied clusters and producing the map $\bar{M}$. **Middle Right** Third, we use $\bar{M}$ to dynamically update the confidence map Γ. **Right** Last, we threshold Γ with hysteresis to generate a binary occupancy map $\hat{M}$.

with initial confidence $\Gamma_0(p) \in [0,1]$. Here, $\beta^-, \beta^+ \in \mathbb{R}_{>0}$ are tunable gains. When $\bar{M}(p) < \sigma$, $\Gamma(p)$ decays exponentially toward 0 at rate β^-. When $\bar{M}(p) \geq \sigma$, $\Gamma(p)$ increases toward 1, with growth scaled by $\beta^+ \bar{M}(p)$, accelerating confidence accumulation in regions with stronger local evidence. The switching parameter $\sigma \in \mathbb{R}_{>0}$ sets the level of spatial consistency required before confidence increases, emphasizing clusters while de-emphasizing isolated detections.

4) Estimated Occupancy Map ($\hat{M}$): Binary thresholding is a natural choice for transforming the confidence map Γ into the estimated occupancy map $\hat{M} : \mathcal{S} \to \{0,1\}$; however, confidence fluctuations near a threshold produce undesirable boundary noise. To mitigate this, we adopt *hysteresis* thresholding, which accounts for the previous occupancy state:

$$\hat{M}(p) := \begin{cases} 1 & \text{if } \Gamma(p) \geq \tau_h, \\ \hat{M}_0(p) & \text{if } \tau_\ell < \Gamma(p) < \tau_h, \\ 0 & \text{if } \Gamma(p) \leq \tau_\ell, \end{cases} \tag{11}$$

where $\tau_h, \tau_\ell \in [0,1]$, $\tau_h > \tau_\ell$ are the upper and lower thresholds respectively, and $\hat{M}_0(p)$ denotes the previous occupancy estimate. This maintains the current occupancy state when confidence lies within (τ_h, τ_ℓ), requiring confidence to cross τ_h to become *occupied* or fall below τ_ℓ to become *free*. The resulting occupancy map $\hat{M}$ provides a spatially and temporally consistent estimate of the true occupancy M. A visual of the entire mapping procedure is depicted in Fig. 2.

B. Geometry-Aware Poisson Safety Function

Consider the safe set $\mathcal{C}_0$ (and its discretized estimate $\hat{M}$). To enforce the forward invariance of $\mathcal{C}_0$, we generate a PSF [27]. We begin by employing a rigid body ROM in which the robot is assumed to have a 2D centroidal position $(x,y) \in \mathbb{R}^2$ and orientation $\theta \in \mathbb{S}^1$. We parameterize the safe set in orientation using the Pontryagin difference:

$$\mathcal{C}(\theta) = \mathcal{C}_0 \ominus \mathcal{R}(\theta), \tag{12}$$

where $\mathcal{R}(\theta) \subset \mathbb{R}^2$ is the set of all points occupied by the robot, in body-centered coordinates, as a function of θ.

We aim to find a safety function $h_0 : \mathbb{R}^2 \times \mathbb{S}^1 \to \mathbb{R}$, whose 0-superlevel set characterizes $\mathcal{C}(\theta) \subset \mathbb{R}^2$ via:

$$\mathcal{C}(\theta) = \{(x,y) \in \mathbb{R}^2 : h_0(x,y,\theta) \geq 0\}. \tag{13}$$

We use a parameterized safe set $\mathcal{C}(\theta)$ to define an orientation-dependent domain Ω_Θ via the lifting operation:

$$\overline{\Omega}_\Theta = \bigcup_{\theta \in \mathbb{S}^1} \mathcal{C}(\theta) \times \{\theta\} \subset \mathbb{R}^2 \times \mathbb{S}^1, \tag{14}$$

and formulate the Dirichlet problem for Poisson's equation:

$$\begin{cases} \dfrac{\partial^2 h_0}{\partial x^2} + \dfrac{\partial^2 h_0}{\partial y^2} = f(x,y,\theta) & \forall (x,y,\theta) \in \Omega_\Theta, \\ h_0(x,y,\theta) = 0 & \forall (x,y,\theta) \in \partial\Omega_\Theta. \end{cases} \tag{15}$$

Solving (15) yields a PSF h_0 that characterizes $\mathcal{C}(\theta)$ in according to (13), and renders $\mathcal{C}(\theta)$ forward invariant with respect to the dynamics describing (x,y,θ).

Table I compares our *custom* mapping algorithm with two baselines: the instantaneous point map (Sec. III-A.1), and the thresholded confidence map (Sec. III-A.3). The former is generated by projecting raw measurements onto a 2D grid, naively treating measurement as ground truth. The latter represents a standard cost map, where cells track occupancy confidence which is updated using new measurements. The comparative baselines are therefore denoted *naive* and *standard*, respectively. In the *static* scenario, each algorithm mapped an unchanging scene. In the *dynamic* scenario, a ball was rolled through the environment. From both experiments, it is clear that, although the dynamic confidence update reduces noise, it does not effectively capture dynamic or sparsely-detected obstacles without spatial smoothing and hysteresis thresholding. Only our approach sufficiently denoises the scene while retaining high ground-truth accuracy.

TABLE I: Comparison of mapping methods. *RMS Noise* (static environment only) was computed using the RMS value of $\frac{\partial h}{\partial t}$ over all grid cells and frame transitions. $\frac{\partial h}{\partial t} = 0$ in static scenes. *Coverage %* is the percent of ground-truth occupied cells, obtained from an overhead video, that are occupied in the map. For each scenario, the best noise/coverage are in **bold**.

Scenario	**Naive** (Projection)		**Standard** (Confidence Map)		**Custom** (Full Algorithm)	
	RMS Noise	**Coverage %**	**RMS Noise**	**Coverage %**	**RMS Noise**	**Coverage %**
Static	0.249	47.40	0.181	39.45	**0.118**	**100**
Dynamic	–	10.93	–	24.61	–	**97.79**

C. Predictive Safety Layer

In our predictive layer, we eliminate the need for a dynamic model by planning in the task space. Specifically, we

plan safe velocity commands for the centroid configuration $\boldsymbol{\chi} = (x, y, \theta) \in \mathbb{R}^2 \times \mathbb{S}^1$ governed by the integrator ROM:

$$\dot{\boldsymbol{\chi}} = \boldsymbol{\mu}, \qquad \boldsymbol{\mu} = (v_x, v_y, \omega) \in \mathbb{R}^3, \tag{16}$$

and rely on the low-level controller to track velocity references on the full-order model [3]–[5], [33]. Under these assumptions, the PSF h_0 is a CBF for the safe set $\mathcal{C}(\theta)$.

Given a nominal command $\boldsymbol{\mu}_{\mathrm{d}}$, we introduce a predictive safety planner for (16) which aims to enforce the forward invariance of a time varying set $\mathcal{C}'(\theta, t) \subset \mathbb{R}^2$ over a finite time horizon using DCBF constraints, while simultaneously minimizing the cumulative deviation from $\boldsymbol{\mu}_{\mathrm{d}}$:

$$\begin{aligned} \min_{\boldsymbol{\nu}_i} \quad & \sum_{i=0}^{N} (\boldsymbol{\mu}_{\mathrm{d}} - \boldsymbol{\nu}_i)^\top \boldsymbol{R} (\boldsymbol{\mu}_{\mathrm{d}} - \boldsymbol{\nu}_i) \\ \text{s.t.} \quad & \boldsymbol{\xi}_0 = \boldsymbol{\chi}, \\ & \boldsymbol{\xi}_{i+1} = \boldsymbol{\xi}_i + \boldsymbol{\nu}_i \delta_t \quad \forall i \in [0, N], \\ & h(\boldsymbol{\xi}_{i+1}, t_{i+1}) \geq \rho h(\boldsymbol{\xi}_i, t_i) \quad \forall i \in [0, N]. \end{aligned} \tag{17}$$

$\boldsymbol{R}$ balances the value of various control actions; the time step $\delta_t \in \mathbb{R}_{>0}$ and the increment $N \in \mathbb{Z}^+$ determine the prediction horizon $T = N\delta_t$; the CBF parameter $\rho \in (0, 1)$ affects dynamic robustness; and the initial $\boldsymbol{\xi}_0$ is constrained to match the state $\boldsymbol{\chi}$ of the ROM.

The time-dependent function h accounts for the uncertain dynamic nature of $\mathcal{C}'(\theta, t)$ along the horizon $t \in [0, T]$ and is coarsely modeled using a first-order extrapolation:

$$h(\boldsymbol{\chi}, t) := h_0(\boldsymbol{\chi}) + \frac{h_0(\boldsymbol{\chi}) - h_{-1}(\boldsymbol{\chi})}{t_0 - t_{-1}}(t - t_0), \tag{18}$$

where h_0 describes the environment at the current time $t_0 \in \mathbb{R}$, and $h_{-1} : \mathbb{R}^2 \times \mathbb{S}^1 \to \mathbb{R}$ is the PSF computed at some previous time $t_{-1} \in \mathbb{R}$ for $t_{-1} < t_0$.

The MPC problem (17)-(18) can be solved with sequential quadratic programming (SQP). The first output in the SQP minimizer $\boldsymbol{\nu}_0^*$ is taken as the planned safe control action $\boldsymbol{\mu}_{\mathrm{p}}$.

D. Real-Time Safety Layer

To synthesize a safe velocity $\boldsymbol{\mu}_{\mathrm{s}}$ that renders $\mathcal{C}'(\theta, t_0)$ forward invariant, we filter the planned action $\boldsymbol{\mu}_{\mathrm{p}}$ from the predictive layer via the *input-to-state safe* (ISSf) CBF-QP:

$$\begin{aligned} \boldsymbol{\mu}_{\mathrm{s}} = \underset{\boldsymbol{\mu} \in \mathbb{R}^3}{\arg\min} \quad & \|\boldsymbol{\mu} - \boldsymbol{\mu}_{\mathrm{p}}\|_2^2 \\ \text{s.t.} \quad & \nabla h \cdot \boldsymbol{\mu} + \frac{\partial h}{\partial t} \geq -\alpha h + \frac{1}{\varepsilon} \|\nabla h\|^2, \end{aligned} \tag{19}$$

where $\frac{1}{\varepsilon}\|\nabla h\|^2$ is a robustness term compensating for the *tracking error* between the ROM and full-order system [5], [33]. We denote the safe ROM command by $\boldsymbol{\mu}_s \in \mathbb{R}^3$, and define the corresponding desired velocity as $\dot{\boldsymbol{\chi}}_{\mathrm{s}} \coloneqq \boldsymbol{\mu}_{\mathrm{s}}$.

This layered architecture interprets safety-critical control as certifying the ability of the full-order robotic system to track the velocity commands $\dot{\boldsymbol{\chi}}_{\mathrm{s}}$ generated by the ROM (16). To demonstrate this rigorously, consider the full-order mechanical system with state $(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^n \times \mathbb{R}^n$ representing all generalized coordinates and their velocities, governed by:

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + G(\mathbf{q}) = \mathbf{u}, \tag{20}$$

where $M(\mathbf{q}) \in \mathbb{R}^{n \times n}$, $C(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{n \times n}$, $G(\mathbf{q}) \in \mathbb{R}^n$ denote the inertia matrix, Coriolis matrix, and gravity terms, respectively, and $\mathbf{u} \in \mathbb{R}^n$ is the generalized input. The ROM state $\boldsymbol{\chi}$ is obtained from the full configuration via an *output map* $\boldsymbol{\chi} = \varphi(\mathbf{q})$ where $\varphi : \mathbb{R}^n \to \mathbb{R}^2 \times \mathbb{S}^1$ represents the relationship between generalized coordinates and ROM pose [4]. Safety for the full-order system is guaranteed provided the true velocity $\dot{\boldsymbol{\chi}} = \frac{\partial \varphi}{\partial \mathbf{q}}\dot{\mathbf{q}}$ tracks the safe velocity $\dot{\boldsymbol{\chi}}_{\mathrm{s}}$ sufficiently well, which we establish in the following:

Theorem 1. *(Full-Order System Safety) Consider the full-order system* (20) *and $T > 0$. Let $h : \mathbb{R}^2 \times \mathbb{S}^1 \times [0, T] \to \mathbb{R}$ be a CBF for the ROM* (16)*, let $\boldsymbol{\chi}_{\mathrm{s}} : [0, T] \to \mathbb{R}^3$ be a safe velocity satisfying* (19) *for some $\varepsilon > 0$, and let $V : \mathbb{R}^n \times \mathbb{R}^n \times \mathbb{R}^3 \times [0, T] \to \mathbb{R}$ be a tracking control Lyapunov function (CLF) satisfying:*

$$V(\mathbf{q}, \dot{\mathbf{q}}, \dot{\boldsymbol{\chi}}_{\mathrm{s}}, t) \geq \beta \|\dot{\boldsymbol{\chi}} - \dot{\boldsymbol{\chi}}_{\mathrm{s}}\|^2, \tag{21}$$

$$\dot{V}(\mathbf{q}, \dot{\mathbf{q}}, \dot{\boldsymbol{\chi}}_{\mathrm{s}}, \ddot{\boldsymbol{\chi}}_{\mathrm{s}}, \mathbf{k}, t) \leq -\lambda V(\mathbf{q}, \dot{\mathbf{q}}, \dot{\boldsymbol{\chi}}_{\mathrm{s}}, t), \tag{22}$$

for some $\beta, \lambda > 0$ and tracking controller $\mathbf{u} = \mathbf{k}(\mathbf{q}, \dot{\mathbf{q}}, \dot{\boldsymbol{\chi}}_{\mathrm{s}}, t)$. Define the barrier candidate $B : \mathbb{R}^n \times \mathbb{R}^n \times [0, T] \to \mathbb{R}$:

$$B(\mathbf{q}, \dot{\mathbf{q}}, t) = h(\varphi(\mathbf{q}), t) - \frac{1}{\mu} V(\mathbf{q}, \dot{\mathbf{q}}, \dot{\boldsymbol{\chi}}_{\mathrm{s}}, t) \tag{23}$$

with $\mu > 0$. If the following condition holds:

$$\lambda \geq \alpha + \frac{\varepsilon \mu}{4\beta}, \tag{24}$$

then B is a CBF for the full-order system (20)*, rendering the time-varying set $\mathcal{C}_{\mathrm{F}}(t) = \{(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^n \times \mathbb{R}^n \mid B(\mathbf{q}, \dot{\mathbf{q}}, t) \geq 0\}$ forward invariant for all $t \in [0, T]$.*

Proof. Taking the time derivative of (23) and noting that $\dot{\boldsymbol{\chi}} = \frac{\partial \varphi}{\partial \mathbf{q}}\dot{\mathbf{q}}$ gives (suppressing function arguments for brevity):

$$\dot{B}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{u}, t) = \frac{\partial h}{\partial \boldsymbol{\chi}} \frac{\partial \varphi}{\partial \mathbf{q}} \dot{\mathbf{q}} + \frac{\partial h}{\partial t} - \frac{1}{\mu} \dot{V}(\mathbf{q}, \dot{\mathbf{q}}_s, \ddot{\mathbf{q}}_s, \mathbf{u}, t) \tag{25}$$

where:

$$\dot{V} = \frac{\partial V}{\partial \mathbf{q}} \dot{\mathbf{q}} + \frac{\partial V}{\partial \dot{\mathbf{q}}} \ddot{\mathbf{q}} + \frac{\partial V}{\partial \dot{\boldsymbol{\chi}}_s} \ddot{\boldsymbol{\chi}}_s + \frac{\partial V}{\partial t},$$

and (20) yields an affine dependence of $\ddot{\mathbf{q}}$ on $\mathbf{u}$. Applying the tracking controller $\mathbf{u} = \mathbf{k}(\mathbf{q}, \dot{\mathbf{q}}, \dot{\boldsymbol{\chi}}_s, t)$ satisfying (22) and lower bounding $\dot{B}$ along the closed loop system yields:

$$\dot{B} \geq \frac{\partial h}{\partial \boldsymbol{\chi}} (\dot{\boldsymbol{\chi}} + \dot{\boldsymbol{\chi}}_s - \dot{\boldsymbol{\chi}}_s) + \frac{\partial h}{\partial t} + \frac{\lambda}{\mu} V \tag{26}$$

$$\geq -\alpha h + \frac{1}{\varepsilon}\|\nabla h\|^2 - \left\| \frac{\partial h}{\partial \boldsymbol{\chi}} \right\| \|\dot{\boldsymbol{\chi}} - \dot{\boldsymbol{\chi}}_s\| + \frac{\lambda}{\mu} V \tag{27}$$

$$\geq -\alpha h - \frac{\varepsilon}{4} \|\dot{\boldsymbol{\chi}} - \dot{\boldsymbol{\chi}}_s\|^2 + \frac{\lambda}{\mu} V \tag{28}$$

$$\geq -\alpha h - \frac{\varepsilon}{4\beta} V + \frac{\lambda}{\mu} V \tag{29}$$

$$= -\alpha B + \frac{1}{\mu} \left(\lambda - \alpha - \frac{\varepsilon \mu}{4\beta} \right) V \tag{30}$$

$$\geq -\alpha B. \tag{31}$$

The first inequality follows from substituting (22) into (25) and adding zero. Then (19) lower bounds this inequality. Completing the squares and lower bounding yields the third inequality. The fourth is reached with (21). Using (23), the last equality is reached, which is bounded with (24). □

Remark 1 (Verification)**.** Although Theorem 1 assumes the existence of a Lyapunov function V and a tracking controller $\mathbf{k}$, explicit expressions are not required to implement our ROM-based safety filter. They serve as tools for verification, i.e., one can certify that a given full-order system tracks the ROM commands by providing bounds on V and stability properties of $\mathbf{k}$. When such bounds are difficult to obtain, (e.g., when V is unknown and $\mathbf{k}$ is implemented as a black-box or learned module in the existing low-level control architecture), Theorem 1 can still be met in practice by initializing α and ε in (19) at small values to ensure conservative safety, and adjusting until adequate performance is attained.

IV. Multistage Safety Filter: Analysis

The previous section established our architecture: PSF synthesis, predictive safe planning, and real-time safety filtering. In this section, we demonstrate how combining these layers in a *multistage* architecture achieves favorable tradeoffs between performance and robustness, compared to either layer operating independently. We examine these tradeoffs in a detailed Pareto optimality analysis and validate them through experiments on quadruped and humanoid hardware.

A. Pareto Optimality Framework

We characterize performance and robustness tradeoffs by examining *task optimality* (deviation from desired behavior) and *safety robustness* (minimum safety margin). We formalize these objectives through two metrics evaluated over the horizon N, spanning an isolated collision avoidance scenario:

$$J_{\text{robustness}} = \min_{i\in[0,N]} h_0(\boldsymbol{\chi}(t_i), t_i),$$
$$J_{\text{optimality}} = \frac{\sum_{i=0}^{N}(\boldsymbol{\nu}_i - \boldsymbol{\mu}_d)^\top R(\boldsymbol{\nu}_i - \boldsymbol{\mu}_d)}{J_{\text{ideal}}},$$

where $J_{\text{robustness}}$ is the minimum value of the PSF h_0 during the maneuver, and $J_{\text{optimality}}$ is the ratio between the total measured control effort of the maneuver ν and the minimum ideal effort J_{ideal}. The ideal cost J_{ideal} is computed by simulating an equivalent scenario with *a priori* known obstacles, providing a benchmark for minimum total safe control effort ν^* for the state-constrained optimal control problem:

$$J_{\text{ideal}} = \sum_{i=0}^{N}(\boldsymbol{\nu}_i^* - \boldsymbol{\mu}_d)^\top R(\boldsymbol{\nu}_i^* - \boldsymbol{\mu}_d).$$

Since these objectives cannot be simultaneously optimized, we introduce a Pareto front to characterize tradeoffs. A solution is *Pareto optimal* if no other solution improves one objective without degrading the other more. The Pareto optimal solution is found by maximizing a *welfare* function, defined as the weighted sum of utilities,

$$\mathbf{z}^* = \arg\max_{\mathbf{z}\in\mathcal{Z}} \left(a_O u_O(\mathbf{z}) + a_R u_R(\mathbf{z})\right),$$

where $\mathbf{z}$ denotes a candidate safety filter architecture, $\mathcal{Z}$ is the set of evaluated architectures, and the utility functions, $u_O(\mathbf{z})$ and $u_R(\mathbf{z})$, quantify the performance of the method $\mathbf{z}$ along the optimality and robustness metrics, respectively. The scalar weights $a_O, a_R > 0$ specify the relative importance of each objective. The weights are set to $a_O = a_R = 0.5$, assigning equal importance to optimality and robustness.

B. Individual Layer Tradeoffs

When operating independently, each safety layer prioritizes different objectives at the expense of the other, thus occupying extreme positions on the Pareto front.

1) Predictive Layer: This layer optimizes a finite-horizon cost, promoting actions that reduce total effort. Long-horizon strategies achieve high optimality by sacrificing immediate safety margins (e.g., side-stepping to avoid obstacles).

2) Real-Time Layer: The CBF-QP enforces instantaneous safety while minimizing deviations from nominal commands. This results in reactive behaviors (e.g., backing away from obstacles), improving robustness by reducing optimality.

3) Multistage Architecture: The multistage design feeds the predictive layer command $\boldsymbol{\mu}_p$ to the real-time CBF-QP. The predictive layer provides long-horizon intent, while the real-time layer enforces the barrier condition by minimally modifying that intent before execution. This composition preserves the predictive layer's efficiency, while improving instantaneous safety guarantees via the real-time CBF-QP, yielding a Pareto-optimal solution between the two extremes.

C. Experimental Validation

Our multistage safety filter was validated using a Unitree Go2 quadruped by comparing three controllers: the isolated predictive filter, the isolated real-time safety filter, and the proposed multistage safety filter. The environment was perceived with two onboard LiDARs. Experiments comprised scenarios where one, two, and three dodgeballs were rolled at the robot. For one-ball experiments, a ball was rolled toward the robot at both 1.0 m/s (slow) and 1.5 m/s (fast). The two-ball experiment introduced a second ball from the opposing side, both at 1.0 m/s. Lastly, three-ball experiments added a ball approaching the front of the robot, with all three rolling at 1.5 m/s. Fig. 3 shows a representative experiment.

Table II reports the optimality and robustness metrics across all experiments. Table II also reports the avoidance success rate and welfare. Utility associated with each metric is computed by normalizing the values and taking the lower bound of its 85% confidence interval. While u_O is computed only from successful trials, u_R incorporates failures by assigning trials a normalized robustness score of 0.

The multistage safety filter consistently achieved the highest success rates and welfare scores for all experiments, supporting its Pareto optimality. The Pareto front is visualized in Fig. 4 for the slow ball scenario using Hotelling's T-squared distribution. The predictive filter achieved high optimality at the cost of robustness. The real-time safety filter had low optimality with high robustness. A state-constrained MPC baseline was also evaluated for this first scenario;

TABLE II: Comparison of safety architectures during collision avoidance experiments. Metrics are reported as a mean and 85% confidence interval (via Student's t-distribution). Within each experimental scenario, the highest lower confidence bound for each metric is in **bold**.

Scenario	Predictive				Real-Time				Multistage			
	Optimality	Robustness	Success %	Welfare	Optimality	Robustness	Success %	Welfare	Optimality	Robustness	Success %	Welfare
1-Ball (Slow)	$\mathbf{0.127 \pm 0.017}$	0.375 ± 0.048	**100**	0.466	0.065 ± 0.005	0.673 ± 0.020	90	0.419	0.105 ± 0.008	$\mathbf{0.682 \pm 0.021}$	**100**	**0.610**
1-Ball (Fast)	$\mathbf{0.201 \pm 0.018}$	0.153*	60	0.414	0.136 ± 0.027	0.203 ± 0.099	30	0.192	0.174 ± 0.022	$\mathbf{0.235 \pm 0.043}$	**100**	**0.485**
2-Ball	$\mathbf{0.182 \pm 0.031}$	0.005 ± 0.032	70	0.325	0.129 ± 0.015	0.070 ± 0.018	40	0.211	0.158 ± 0.028	$\mathbf{0.087 \pm 0.014}$	**90**	**0.377**
3-Ball	$\mathbf{0.077 \pm 0.023}$	-0.082 ± 0.0411	70	0.085	0.058 ± 0.011	$\mathbf{0.007 \pm 0.072}$	50	0.070	0.055 ± 0.007	-0.018 ± 0.055	**80**	**0.101**
Humanoid	0.053 ± 0.008	0.399 ± 0.083	60	0.141	0.057 ± 0.004	0.399 ± 0.069	**100**	0.327	$\mathbf{0.065 \pm 0.008}$	$\mathbf{0.530 \pm 0.048}$	**100**	**0.505**

* For these experiments, h could not be accurately measured during the ball's closest approach. Thus, the minimum observed value of h is reported as a conservative upper bound. The true value for each experimental trial was lower, but unmeasurable.

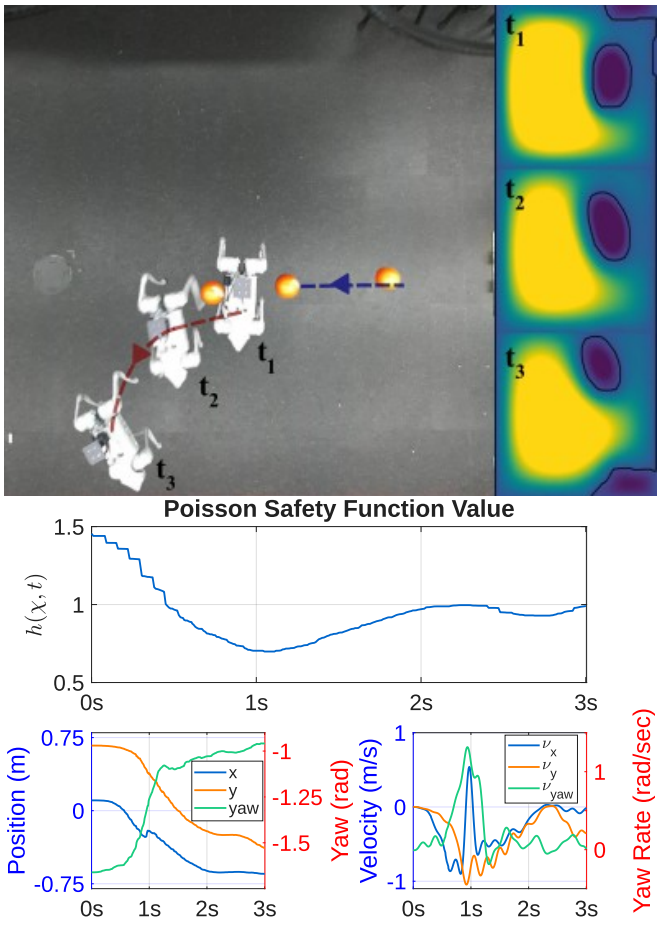

Fig. 3: Dynamic Collision Avoidance Trial. **Top left** Quadruped avoiding a ball from right to left at three times. **Top right** Robocentric PSFs h at each time instance. **Middle** The safety function value $h(x, y, \theta, t)$ used to compute $J_{\text{robustness}}$. **Bottom Left** Measured position and yaw angle of the quadruped. **Bottom Right** Filtered velocity commands used to compute $J_{\text{optimality}}$. Footage of experiment trials is presented in the supporting video.

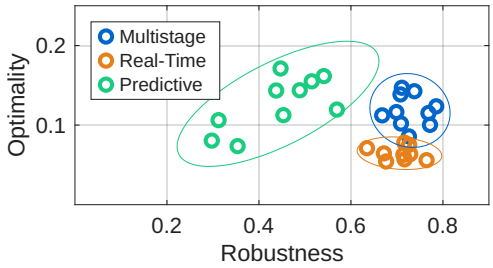

Fig. 4: Analysis of Avoidance Experiment. Evaluated optimality and robustness metrics of collision avoidance with ball speeds of 1.0 m/s, with a confidence ellipse of the population mean. The results show tradeoffs in the metrics, forming an approximation of a Pareto front.

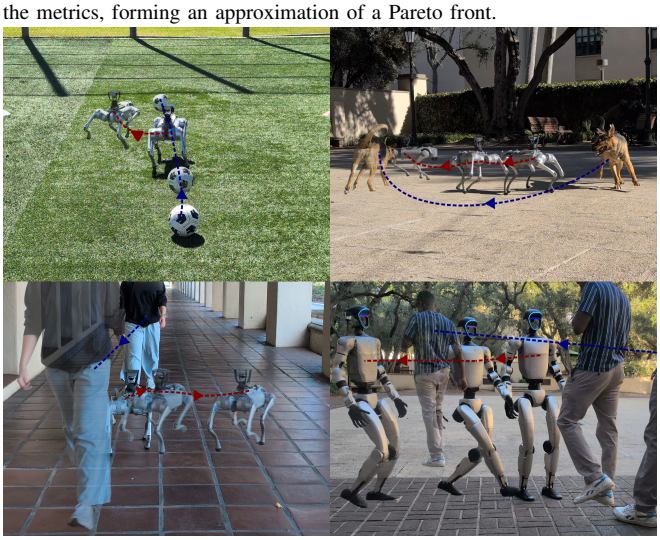

Fig. 5: Collision Avoidance in Practical Settings. **Top Left** Quadruped avoids a soccer ball on a turf field. **Top Right** Quadruped avoids a persuant dog. **Bottom Left** Quadruped avoids a pedestrian. **Bottom Right** Humanoid avoids a human. Obstacle trajectories are indicated in blue, while avoidance trajectories are shown in red. Footage is available in the supplemental video.

however, it lacked the necessary robustness for dynamic collision avoidance, failing all 10 trials. Consequently, the performance metrics could not be meaningfully computed.

As avoidance difficulty increased, failures became more frequent. The predictive filter fails due to its low control rate, limiting its robustness in dynamic environments. Meanwhile, failures of the real-time filter arise from its myopic nature in conjunction with model mismatch (the single integrator model does not account for input limits). At higher speeds, obstacles exceed input limits and collide with the robot.

Finally, because our ROM-based framework does not make assumptions on the underlying full-order model, the control architecture can be deployed on additional robotic platforms. Thus, we implemented it on the Unitree G1 humanoid robot using only the built-in LiDAR for perception. A yoga ball was rolled toward the humanoid robot at 1.1 m/s, and the multistage safety filter again achieved the highest success rate and welfare score (Table II), verifying the Pareto optimality tradeoffs persist across robotic platforms. Our multistage architecture also outperformed the other two methods in optimality and robustness metrics.

D. Generality of Multistage Safety Architecture

To demonstrate the generality of our safety architecture across various environments and obstacles, we deployed it in multiple unstructured real-world scenarios. Examples are depicted in Fig. 5. Fig. 5 (top left) shows the Unitree Go2 quadruped successfully avoiding a rolling ball on a slippery

turf field. Fig. 5 (top right) shows the quadruped safely avoiding an approaching dog, (bottom left) shows safe collision avoidance as the quadruped and human approach one another, and (bottom right) shows the humanoid successfully avoiding collisions with an approaching person, demonstrating applicability across different dynamic obstacles.

V. CONCLUSIONS

We have presented a multistage architecture for dynamic collision avoidance on multiple robotic platforms. We provided an occupancy mapping algorithm to synthesize safety constrainta via PSFs, which we incorporated into two safety layers: 1) a predictive MPC layer, and 2) a real-time CBF-QP safety filter. We validated the optimality and robustness of this architecture via Pareto analysis on quadruped and humanoid platforms in dynamic environments. Future work includes extensions to high-dimensional systems, multi-robot systems, and integration of semantic scene understanding.

REFERENCES

[1] A. Ames, X. Xu, J. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," vol. 62, no. 8, pp. 3861–3876, 2017.

[2] A. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," pp. 3420–3431, June 2019.

[3] T. G. Molnar, R. K. Cosner, A. W. Singletary, W. Ubellacker, and A. D. Ames, "Model-free safety-critical control for robotic systems," *Robotics & Automation Letters*, vol. 7, no. 2, pp. 944–951, 2022.

[4] M. H. Cohen, N. Csomay-Shanklin, W. D. Compton, T. G. Molnar, and A. D. Ames, "Safety-critical controller synthesis with reduced-order models," in *2025 American Control Conference (ACC)*, pp. 5216–5221, 2025.

[5] M. H. Cohen, T. G. Molnar, and A. D. Ames, "Safety-critical control for autonomous systems: Control barrier functions via reduced-order models," *Annual Reviews in Control*, vol. 57, p. 100947, 2024.

[6] M. H. Cohen, R. K. Cosner, and A. D. Ames, "Constructive safety-critical control: Synthesizing control barrier functions for partially feedback linearizable systems," *IEEE Control Systems Letters*, 2024.

[7] G. Bahati, R. K. Cosner, M. H. Cohen, R. M. Bena, and A. D. Ames, "Control barrier function synthesis for nonlinear systems with dual relative degree," *2025 IEEE 64th Conference on Decision and Control (CDC)*, 2025.

[8] G. Bahati, R. M. Bena, and A. D. Ames, "Dynamic safety in complex environments: Synthesizing safety filters with poisson's equation," in *Robotics: Science and Systems*, 2025.

[9] M. F. Reis, A. P. Aguiar, and P. Tabuada, "Control barrier function-based quadratic programs introduce undesirable asymptotically stable equilibria," *IEEE Control Systems Letters*, vol. 5, no. 2, pp. 731–736, 2020.

[10] P. Mestres, Y. Chen, E. Dall'Anese, and J. Cortés, "Control barrier function-based safety filters: characterization of undesired equilibria, unbounded trajectories, and limit cycles," *arXiv preprint arXiv:2501.09289*, 2025.

[11] F. Borrelli, A. Bemporad, and M. Morari, *Predictive control for linear and hybrid systems*. Cambridge University Press, 2017.

[12] Ren, Yunfan et al., "Safety-assured high-speed navigation for MAVs," vol. 10, no. 98, p. eado6187.

[13] J. Zeng, B. Zhang, and K. Sreenath, "Safety-critical model predictive control with discrete-time control barrier function," in *2021 American Control Conference (ACC)*, pp. 3882–3889, 2021.

[14] H. Ma, X. Zhang, S. E. Li, Z. Lin, Y. Lyu, and S. Zheng, "Feasibility enhancement of constrained receding horizon control using generalized control barrier function," in *4th IEEE International Conference on Industrial Cyber-Physical Systems (ICPS)*, pp. 551–557, 2021.

[15] J. Zeng, Z. Li, and K. Sreenath, "Enhancing feasibility and safety of nonlinear model predictive control with discrete-time control barrier functions," in *2021 60th IEEE Conference on Decision and Control (CDC)*, pp. 6137–6144, IEEE, 2021.

[16] S. Liu, J. Zeng, K. Sreenath, and C. A. Belta, "Iterative convex optimization for model predictive control with discrete-time high-order control barrier functions," *arXiv preprint arXiv:2210.04361*, 2022.

[17] K. P. Wabersich and M. N. Zeilinger, "Predictive control barrier functions: Enhanced safety mechanisms for learning-based control," *IEEE Transactions on Automatic Control*, vol. 68, no. 5, pp. 2638–2651, 2022.

[18] H. Abdi, P. Zhao, N. Hovakimyan, and R. Ghabcheloo, "Model predictive control barrier functions: Guaranteed safety with reduced conservatism and shortened horizon," in *2024 American Control Conference (ACC)*, pp. 1652–1657, IEEE, 2024.

[19] K. Long, C. Qian, J. Cortés, and N. Atanasov, "Learning barrier functions with memory for robust safe navigation," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 4931–4938, 2021.

[20] Jian Zhuozhu et al., "Dynamic control barrier function-based model predictive control to safety-critical obstacle-avoidance of mobile robot," *arXiv preprint arXiv:2209.08539*, 2022.

[21] C. Dawson, B. Lowenkamp, D. Goff, and C. Fan, "Learning safe, generalizable perception-based hybrid control with certificates," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 1904–1911, 2022.

[22] M. De Sa, P. Kotaru, and K. Sreenath, "Point cloud-based control barrier function regression for safe and efficient vision-based control," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 366–372, 2024.

[23] S. Keyumarsi, M. W. S. Atman, and A. Gusrialdi, "Lidar-based online control barrier function synthesis for safe navigation in unknown environments," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1043–1050, 2024.

[24] P. Tooranjipour and B. Kiumarsi, "Lidar-based model predictive control using control barrier functions," in *2025 American Control Conference (ACC)*, pp. 315–322, 2025.

[25] L. Yang, B. Werner, R. K. Cosner, D. Fridovich-Keil, P. Culbertson, and A. D. Ames, "Shield: Safety on humanoids via cbfs in expectation on learned dynamics," *arXiv preprint arXiv:2505.11494*, 2025.

[26] J. Liu, M. Li, J. W. Grizzle, and J.-K. Huang, "Clf-cbf constraints for real-time avoidance of multiple obstacles in bipedal locomotion and navigation," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 10497–10504, 2023.

[27] R. M. Bena, G. Bahati, B. Werner, R. K. Cosner, L. Yang, and A. D. Ames, "Geometry-aware predictive safety filters on humanoids: From poisson safety functions to cbf constrained mpc," *IEEE-RAS 24th International Conference on Humanoid Robots (Humanoids)*, 2025.

[28] J.-R. Chiu, J.-P. Sleiman, M. Mittal, F. Farshidian, and M. Hutter, "A collision-free mpc for whole-body dynamic locomotion and manipulation," in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 4686–4693, 2022.

[29] M. Gaertner, M. Bjelonic, F. Farshidian, and M. Hutter, "Collision-free mpc for legged robots in static and dynamic scenes," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8266–8272, IEEE, 2021.

[30] X. Zihao, C. Hao, C. Wang, K. Sima, F. Shi, and J. Song Dong, "Rebot: Reflexive evasion robot for instantaneous dynamic obstacle avoidance," *arXiv preprint arXiv:2508.06229*, 2025.

[31] R. Grandia, A. J. Taylor, A. D. Ames, and M. Hutter, "Multi-layered safety for legged robots via control barrier functions and model predictive control," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8352–8358, IEEE, 2021.

[32] T. G. Molnar, R. K. Cosner, A. W. Singletary, W. Ubellacker, and A. D. Ames, "Model-free safety-critical control for robotic systems," vol. 7, no. 2, pp. 944–951, 2022.

[33] W. D. Compton, M. H. Cohen, and A. D. Ames, "Learning for layered safety-critical control with predictive control barrier functions," in *Proceedings of the 7th Annual Learning for Dynamics & Control Conference*, vol. 283, pp. 153–165, 2025.

[34] S. A. Esteban, M. H. Cohen, A. B. Ghansah, and A. D. Ames, "A layered control perspective on legged locomotion: Embedding reduced order models via hybrid zero dynamics," *arXiv preprint arXiv:2509.00294*, 2025.

[35] L. Perko, *Differential equations and dynamical systems*, vol. 7. Springer Science & Business Media, 2013.

[36] R. K. Cosner, *Dynamic Safety Under Uncertainty: A Control Barrier Function Approach*. PhD thesis, California Inst. of Tech., 2025.